

Axon Tickets Event Readiness Plan

Prepared for the June 11, 2026 event window

This plan assumes the platform has real performance risk today, but that the event is too close for broad architectural changes. The operating principle is simple: protect execution first, then fix the architecture after the event.

Executive Decision

Do not implement major performance refactors before June 11. Enter a stability freeze now.

Operational Priority

Protect customer registration, proof submission, admin verification, and check-in above everything else.

Why This Is The Right Call

- Observed public API latency is already elevated, especially on event listing endpoints.
- The highest-value fixes sit in core request paths and data-loading logic, which are high-blast-radius changes.
- Changing core read behavior, cache strategy, or admin query architecture a few days before an event increases regression risk.
- From a CTO standpoint, event execution risk is now more important than technical elegance.

What Gets Frozen Now

- No major API query rewrites on public event, admin transactions, attendees, analytics, or auth paths.
- No caching strategy rewrites on customer pages.
- No deployment shape changes to the API or Vercel setup.
- No non-essential UI changes to registration, payment, verification, or check-in flows.

Critical Paths To Protect

Priority	Path	Why It Matters
1	Customer event page	Entry point for discovery and conversion.
2	Customer registration flow	Direct revenue path; cannot feel broken or unstable.
3	Payment proof upload	Manual-payment flow depends on this completing cleanly.
4	Admin verification queue	Back-office throughput determines whether buyers get confirmed.
5	Admin check-in	Event-day operations fail if this becomes slow or unreliable.

Non-Critical Features During Event Window

- Real-time analytics
- CSV exports
- Historical transactions browsing
- Non-essential admin browsing across multiple tabs

June 7–11 Operating Plan

June 7

1. Declare major-change freeze.
2. Assign one technical owner for event readiness.
3. Set up monitoring for homepage, event page, API health, registration, verification, and check-in.
4. Inform staff that analytics and exports are secondary during event hours.

June 8

1. Run a rehearsal across event page, registration, proof upload, verification, and check-in.
2. Capture approximate response times and failure points.
3. Decide fallback operating rules if admin tools slow down.

June 9

1. Train staff on which pages to use and which to avoid during peak load.
2. Prepare a simple incident chain: owner, comms, verification, support.
3. Reduce unnecessary browser tabs and duplicate admin usage.

June 10

1. No risky deploys.
2. Do one final smoke test on critical flows only.
3. Confirm support contacts and escalation owner.
4. Confirm event data is clean and ready.

Event-Day Rules

- One person monitors health and latency; do not split ownership.
- Verification staff use verification tools only.
- Check-in staff use check-in tools only.
- Avoid analytics, exports, and unnecessary admin browsing during peak periods.
- If the system slows down, protect check-in and registration first.

Fallback Response If Performance Degrades

1. Immediately stop analytics usage.
2. Immediately stop CSV exports.
3. Reduce concurrent admin traffic to verification and check-in only.
4. Keep customer registration and check-in as top priority.
5. Shift to a tighter manual operating workflow if needed.

Post-Event Engineering Priorities

1. Remove write-on-read behavior from event listing flows.
2. Rebuild admin transactions and attendees to paginate in the database.
3. Fix public caching strategy for homepage and event discovery.
4. Add endpoint timing instrumentation and baseline dashboards.
5. Review serverless cold-start and API packaging overhead.

Execution date for this work should begin immediately after June 11, not before.

CEO Alignment Message

Hi [CEO Name], quick alignment on the platform ahead of the June 11 event.

I reviewed the current performance risk directly, and there is real slowness on both the customer and admin side. The issue is not imaginary, and I do not want us to downplay it. That said, the deeper fixes sit in core API and admin data-loading paths. Those changes are important, but they are not low-risk changes to push a few days before a live event.

My recommendation is firm: we should not implement the major performance refactors before June 11. From a CTO risk perspective, changing core request behavior right now creates more operational risk than leaving the current version in place for the event.

What I recommend instead is a stability-first plan:

1. Freeze major code changes now.
2. Closely monitor the critical flows only: event page, registration, payment proof upload, verification, and check-in.
3. Limit non-essential admin activity during peak periods, especially analytics and exports.
4. Run the event on a controlled operating plan, then begin the deeper optimization work immediately after June 11.

So the position is clear: yes, the performance issue is real; no, this is not the right week to refactor the platform deeply; and yes, we already have a practical plan to reduce risk and protect event execution.

If you want, I can walk you through the event readiness plan and the post-event remediation sequence.